

Escenario:

En el mundo digital actual, las compras online se han convertido en la norma. La industria del libro no se ha quedado atrás, y las plataformas de venta online de libros están en auge. En este escenario, la empresa "Bibliópolis" busca consolidarse como líder en el mercado, ofreciendo una experiencia de compra de libros online fácil, eficiente y atractiva.

Bibliópolis necesita desarrollar una plataforma web que permita a los usuarios comprar libros de forma online. La plataforma debe ser fácil de usar, intuitiva y segura. Además, debe ofrecer un amplio catálogo de libros, información detallada de cada libro, opciones de búsqueda y filtrado, métodos de pago seguros y envío rápido y confiable.

Objetivo:

Identifica los objetos que serían necesarios para modelar este sistema de reclutamiento.

Usuario:

- id: identificador único del usuario, generado automáticamente
- nombre: nombre del usuario
- correoElectronico: correo electrónico del usuario, único
- contrasena: String contraseña del usuario
- direccion: String dirección de envío del usuario

Libro

- isbn: ISBN del libro, obligatorio, único)
- titulo: título del libro, obligatorio)
- autor: autor del libro, obligatorio)
- editorial: editorial del libro)
- genero: género literario del libro)
- precio: precio del libro
- descripcion: descripción del libro
- imagen: ruta a la imagen de portada del libro
- stock: cantidad de libros disponibles en stock

Pedido

- id: identificador único del pedido, generado automáticamente
- usuario: usuario que realizó el pedido

- fechaPedido: fecha en la que se realizó el pedido
- estado: estado actual del pedido: "pendiente", "en proceso", "enviado", "entregado"
- total: precio total del pedido (Calculado)
- items: lista de ítems que componen el pedido

ItemPedido

- libro: libro que forma parte del pedido
- cantidad: cantidad de libros del mismo tipo en el pedido

Relaciones

- Un Usuario puede realizar muchos Pedidos.
- Un Pedido pertenece a un único Usuario.
- Un Pedido puede tener muchos ItemPedidos.
- Un ItemPedido pertenece a un único Pedido.
- Un ItemPedido está asociado a un único Libro.
- Un Libro puede estar asociado a varios ItemPedidos.

Actividades:

1. Crea un nuevo proyecto utilizando nestjs llamado "bibliopolis-backend"
2. Crea un nuevo repositorio en github llamado "bibliopolis-backend"
3. Sincroniza tu repositorio local con el repositorio de github
4. Identifica las clases necesarias según el escenario planteado y por cada clase define los atributos especificando el tipo de dato necesario.
5. Configura swagger en tu proyecto
 - a. título : API Biblioteca
 - b. descripción : Esta api describe la plataforma de libros
 - c. versión : 1.0.0
 - d. tag : biblioteca-api
6. Crea las siguientes clases y sus relaciones en la **carpeta models**
 - a. Usuario
 - b. Libro
 - c. Pedido

- d. ItemPedido
- 7. Haz un commit con el mensaje "Modelo creado" y luego haz un push al repositorio de github.
- 8. Crea una nueva rama en tu repositorio local llamada "**feature_usuarios**"
- 9. Crea el controlador para **Usuarios** que permita realizar las siguientes acciones

Acción	Método HTTP	Path
Registrar un nuevo usuario	POST	/usuarios
Obtener un usuario según su id	GET	/usuarios/:id
Obtener todos los usuarios	GET	/usuarios
Eliminar un usuario según su id	DELETE	/usuarios/:nombreUsuario

- 10. Crea un servicio para almacenar los datos de los **usuarios** y que permita dar soporte a las acciones que maneja el controlador de usuarios
 - a. Registrar un nuevo usuario (Verificar si existe el usuario según el correo ingresado)
 - b. Obtener un usuario según su id, en caso de que el usuario no exista devolver el código 404.
 - c. Obtener todos los usuarios (**Excluir la password en la lista**)
 - d. Eliminar un usuarios según su id
- 11. Haz un commit con el mensaje "**Módulo de usuarios**" y luego haz un push al repositorio de github usando la rama "**feature_usuarios**".
- 12. Haz un pull request hacia la rama master en github
- 13. En el repositorio local cambiarse a la rama 'main'
- 14. Sincronizar los cambios con el repositorio remoto
- 15. Crear una nueva rama llamada "**feature_libros**"
- 16. Crea el controlador para **Libros** que permita realizar las siguientes acciones

Acción	Método HTTP	Path
--------	-------------	------

Crear un nuevo libro	POST	/libros
Obtener un libro según su ISBN	GET	/libros/:isbn
Obtener todos los libros y permitir filtrar por autor y/o género	GET	/libros?autor=&&genero=
Eliminar un libro según su ISBN	DELETE	/libros/:isbn

17. Crea un servicio para almacenar los **datos de libros** y que permita dar soporte a las acciones que maneja el controlador de **libros**
 - a. crear un nuevo libro, debe verificar que el ISBN no exista.
 - b. Obtener un libro según su ISBN
 - c. Obtener todas los libros y permitir filtrar por autor y/o género (Si no se envían los filtros de autor o género debe devolver todos los libros,)
 - d. Eliminar un libro según su ISBN
18. Haz un commit con el mensaje **“Módulo de libros”** y luego haz un push al repositorio de github usando la rama **“feature_libros”**.
19. Repite los pasos 11, 12, 13
20. Crear una nueva rama llamada **“feature_pedidos”**
21. Crea el controlador para **Pedidos** que permita realizar las siguientes acciones
 - a. crear un nuevo pedido
 - b. Obtener un pedido según su Id
 - c. Obtener todos los pedidos y permitir filtrar por estado y/o usuario
 - d. Modificar el estado de un pedido según su id

Acción	Método HTTP	Path
Crear un nuevo pedido	POST	/pedidos
Obtener un pedido según su Id	GET	/pedidos/:id
Obtener todos los pedidos y permitir filtrar por estado y/o usuario	GET	/pedidos?estado=&&usuario=
Modificar el estado de un pedido según su id	PUT	/pedidos/:id + {body}

22. Crea un servicio para almacenar los **datos de pedidos** y que permita dar soporte a las acciones que maneja el controlador de **pedidos**
- a. crear un nuevo pedido
 - i. La fechaPedido debe ser igual a la fecha actual del sistema
 - ii. El total se debe calcular en base al precio de cada libro en el pedido y su cantidad.
 - iii. Se debe validar que exista stock para cada uno de los libros en el pedido.
 - b. Obtener un pedido según su Id
 - c. Obtener todos los pedidos y permitir filtrar por estado y/o usuario
 - d. Modificar el estado de un pedido según su id
 - i. Si el estado actual del pedido es "pendiente" solo debe permitir el valor "en proceso"
 - ii. Si el estado actual del pedido es "en proceso" solo debe permitir el valor "enviado"
 - iii. Si el estado actual del pedido es "enviado", sólo debe permitir el valor "entregado"
 - iv. Si el estado actual del pedido es "entregado" no se puede modificar
 - v. Si no debe devolver el 400 con el mensaje "estado incorrecto"
23. Haz un commit con el mensaje "**Módulo de Pedidos**" y luego haz un push al repositorio de github usando la rama "**feature_pedidos**".